

LIVRET D'EXCELLENCE TOUT-EN-UN

SÉRIE SCIENCES DE L'INFORMATIQUE (BAC TUNISIE)

CHAPITRE 1 : ENREGISTREMENTS & FICHIERS

mathbacai.com
L'excellence au bout des doigts

Ce livret d'excellence haut de gamme regroupe :

- **Fiche 1 : Le Cours Complet**
(Modélisation théorique, déclarations algorithme & Pascal, manipulations avancées, cycle de vie complet, gestion des erreurs d'Entrée/Sortie, fichiers d'accès direct et fichiers textes).
- **Fiche 2 : Le Recueil de Questions-Réponses**
(40 Flashcards de révision active pour l'écrit ou l'oral du Baccalauréat avec questions et réponses).
- **Fiche 3 : Quiz d'Évaluation & Fiches Mémo**
(10 questions types QCM/Vrai-Faux avec correction chirurgicale hyper-détaillée, tableau de synthèse universel et mémo des erreurs classiques).

Année Scolaire : 2026 - 2027

Contents

1	Fiche 1 : Le Cours Complet	3
1.1	La Structure Enregistrement (Record)	3
1.1.1	Introduction et Limitations des Tableaux	3
1.1.2	Déclaration et Correspondance Algorithme - Pascal	3
1.1.3	Déclarations Imbriquées (Enregistrements Imbriqués)	3
1.1.4	Accès aux champs et manipulation	4
1.1.5	Structure d'optimisation : L'instruction Avec .. Faire	4
1.1.6	Tableaux d'Enregistrements	5
1.2	La Structure Fichier (File)	6
1.2.1	Introduction et Persistance des données	6
1.2.2	Définition d'un Fichier	6
1.2.3	Déclaration de Fichier	6
1.2.4	Le Cycle de Vie d'un Fichier	6
1.2.5	Détail des Fonctions Standard de Gestion des Fichiers	6
1.2.6	Programmation Défensive : Contrôle des Erreurs d'Entrée/Sortie (E/S)	6
1.3	Fichiers à Accès Direct & Fichiers Textes	8
1.3.1	Fichiers à Accès Direct	8
1.3.2	Algorithme standard d'ajout de données (Append sur Fichier Binaire)	8
1.3.3	Fichiers Textes (Text)	8
2	Fiche 2 : Recueil de Questions-Réponses	9
3	Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo	16
3.1	Le Quiz d'Évaluation (10 Questions Types Bac)	16
3.2	Mémento de l'Excellence (Tableau Récapitulatif Universel)	19
3.3	Les 5 Pièges Classiques du Bac à éviter absolument	19

Chapter 1

Fiche 1 : Le Cours Complet

1.1 La Structure Enregistrement (Record)

1.1.1 Introduction et Limitations des Tableaux

Les structures de données simples étudiées précédemment (comme les types scalaires) et les tableaux possèdent une limitation majeure : l'**homogénéité** des données. Un tableau ne peut contenir que des éléments de même type.

Pour modéliser une entité réelle complexe (comme un étudiant avec un nom, une moyenne, une date de naissance), nous avons besoin d'associer des données de types hétérogènes. C'est l'objectif de la structure **Enregistrement** (appelée **Record** en Pascal).

1.1.2 Déclaration et Correspondance Algorithmique - Pascal

La déclaration d'un enregistrement se fait d'abord par la définition d'un nouveau type personnalisé, suivi de la déclaration de la variable utilisant ce type.

Déclaration en Algorithmique (Nouveau Type)

```
Type complexe = Enregistrement
    p_réelle : Réel
    p_imaginaire : Réel
Fin complexe
```

Traduction en Pascal

```
TYPE complexe = Record
    p_reelle : Real;
    p_imaginaire : Real;
End;
```

1.1.3 Déclarations Imbriquées (Enregistrements Imbriqués)

Un champ d'un enregistrement peut lui-même être un autre enregistrement. C'est ce qu'on appelle la structure d'enregistrement imbriquée. Par exemple, pour stocker une date de naissance dans la fiche d'un étudiant :

Déclaration Algorithmique (Enregistrements Imbriqués)

```
Type Date = Enregistrement
    jour : Entier
    mois : Chaîne
    année : Entier
```

Fin Date

Type Fiche = Enregistrement

nom : Chaîne

prénom : Chaîne

date_nais : Date

moyenne : Réel

Fin Fiche

Traduction en Pascal (Enregistrements Imbriqués)

```
TYPE Date = Record
  jour : Integer;
  mois : String;
  annee : Integer;
End;
```

```
TYPE Fiche = Record
  nom : String;
  prenom : String;
  date_nais : Date;
  moyenne : Real;
End;
```

1.1.4 Accès aux champs et manipulation

L'accès à un champ d'un enregistrement se fait à l'aide de la **notation pointée** : l'identificateur de la variable suivi d'un point (.) puis de l'identificateur du champ recherché.

Syntaxe d'Accès aux champs

variable_enregistrement . champ

Exemple de manipulation en Pascal :

Exemple d'accès en Pascal

```
VAR etudiant : Fiche;
Begin
  etudiant.nom := 'Gharbi';
  etudiant.prenom := 'Ahmed';
  etudiant.date_nais.jour := 15;
  etudiant.date_nais.mois := 'Mars';
  etudiant.date_nais.annee := 2008;
  etudiant.moyenne := 15.75;
End;
```

1.1.5 Structure d'optimisation : L'instruction Avec .. Faire

Pour simplifier l'écriture et éviter la répétition de l'identificateur de la variable, l'instruction **Avec .. Faire (With .. Do)** permet d'ouvrir un bloc où les champs sont directement accessibles sans la notation pointée.

Syntaxe Algorithmique**Avec** étudiant **Faire**

nom ← "Gharbi"

prénom ← "Ahmed"

moyenne ← 15.75

Fin Avec**Traduction en Pascal**

With etudiant Do Begin

nom := 'Gharbi';

prenom := 'Ahmed';

moyenne := 15.75;

End;

1.1.6 Tableaux d'Enregistrements

On peut déclarer un tableau dont les éléments sont de type enregistrement. Cela permet de modéliser une collection ou une base de données locale en mémoire vive.

Déclaration d'un Tableau d'Enregistrements en Pascal

TYPE Tab_Fiches = Array[1..100] Of Fiche;

VAR classe : Tab_Fiches;

1.2 La Structure Fichier (File)

1.2.1 Introduction et Persistance des données

Toutes les structures de données étudiées jusqu'à présent (variables simples, tableaux, enregistrements) résident en **mémoire vive (RAM)**. Cette mémoire est volatile : elle s'efface dès la fermeture du programme ou l'extinction de la machine.

Pour conserver les données de manière définitive (persistance), l'informatique utilise la **mémoire secondaire** (disque dur, SSD, clé USB) sous forme de **Fichiers**.

1.2.2 Définition d'un Fichier

Un fichier est un ensemble de données de même type stocké sur un support physique permanent. Il est structuré en composants linéaires (enregistrements logiques). Contrairement au tableau, sa taille n'est pas limitée par la mémoire de travail et n'a pas besoin d'être déclarée à l'avance.

1.2.3 Déclaration de Fichier

Un fichier binaire (composé d'éléments typés) se déclare en définissant le type de ses composants :

Déclaration de Fichier (Algorithmique)

```
Type F_Entiers = Fichier de Entiers
Objet Global :
  F : F_Entiers
```

Traduction en Pascal

```
TYPE F_Entiers = File Of Integer;
VAR F : F_Entiers;
```

1.2.4 Le Cycle de Vie d'un Fichier

Pour manipuler correctement un fichier binaire ou texte dans un programme, il faut obligatoirement respecter l'enchaînement de 4 étapes majeures :



1.2.5 Détail des Fonctions Standard de Gestion des Fichiers

Opération	Syntaxe Algorithmique	Syntaxe Pascal	Rôle / Effet
Association	Associer(F, "élèves.dat")	Assign(F, 'élèves.dat');	Lier logique & physique
Création	Recréer(F)	ReWrite(F);	Écraser ou créer à neuf
Ouverture	Ouvrir(F)	ReSet(F);	Ouvrir en lecture/écriture
Fermeture	Fermer(F)	Close(F);	Enregistrer et libérer
Fin de Fichier	Fin_fichier(F)	Eof(F)	Booléen retournant Vrai si fin

Table 1.1: Opérations physiques et logiques sur fichiers

1.2.6 Programmation Défensive : Contrôle des Erreurs d'Entrée/Sortie (E/S)

Par défaut, si une instruction d'E/S échoue (par exemple, l'ouverture avec ReSet d'un fichier inexistant), le programme Pascal s'arrête brutalement sur un bug d'exécution d'E/S (I/O Error). La programmation

défensive permet d'éviter cela en désactivant le contrôle système pour gérer l'anomalie dans le code.

Code Pascal de Sécurisation d'Ouverture

```
{ $I- }                                     { Désactive l'arrêt sur erreur d'E/S }  
ReSet(F);  
{ $I+ }                                     { Réactive les vérifications d'E/S }  
IF IOResult <> 0 THEN Begin  
    WriteLn('Le fichier n'existe pas.  Creation d'un nouveau...');  
    ReWrite(F);  
End;
```

1.3 Fichiers à Accès Direct & Fichiers Textes

1.3.1 Fichiers à Accès Direct

Un fichier à accès direct (ou fichier binaire structuré) permet d'accéder directement à n'importe quel enregistrement grâce à sa position (index), sans lire les précédents composants. Les enregistrements sont indexés de 0 à $\text{FileSize}(\text{F}) - 1$.

Fonctions Clés de l'Accès Direct

- **Pointer (Seek in Pascal)** : Déplace le pointeur sur le composant numéro N .
Syntaxe : $\text{Pointer}(\text{F}, N) \rightarrow \text{Seek}(\text{F}, N)$;
- **Taille_fichier (FileSize)** : Retourne le nombre total de composants du fichier.
Syntaxe : $T \leftarrow \text{Taille_fichier}(\text{F}) \rightarrow T := \text{FileSize}(\text{F})$;
- **Position_fichier (FilePos)** : Retourne l'index courant du pointeur.
Syntaxe : $P \leftarrow \text{Position_fichier}(\text{F}) \rightarrow P := \text{FilePos}(\text{F})$;

1.3.2 Algorithme standard d'ajout de données (Append sur Fichier Binaire)

Pour ajouter un nouvel élément à la fin d'un fichier binaire d'accès direct, la procédure standard consiste à se positionner à l'index correspondant à la taille totale du fichier (qui équivaut au premier emplacement virtuel disponible).

Algorithme de Positionnement de Fin

```
Pointer(F, Taille_fichier(F))
Ecrire(F, nouvel_élément)
```

Équivalent Pascal d'Ajout

```
Seek(F, FileSize(F));
Write(F, nouvel_element);
```

1.3.3 Fichiers Textes (Text)

Un fichier texte (**Text** ou **Texte**) est une suite séquentielle pure de caractères structurée en lignes d'une longueur variable. Chaque ligne se termine par un caractère invisible de fin de ligne appelé **Retour Chariot** / saut de ligne (CR-LF).

Propriétés et Fonctions Spécifiques aux Fichiers Textes

- **Type Texte (Text)** : Déclaration spécifique. Pascal : `VAR Ft : Text;`
- **Fin_ligne (EoLn)** : Fonction booléenne retournant Vrai si le pointeur est positionné sur la marque de fin de ligne.
- **Ajouter (Append)** : Procédure spécifique d'ouverture en mode écriture à la suite (les données existantes sont conservées). Pascal : `Append(Ft);`
- **Lire_nl (ReadLn) & Ecrire_nl (WriteLn)** : Lecture ou écriture d'une chaîne complète avec saut de ligne automatique.

Chapter 2

Fiche 2 : Recueil de Questions-Réponses

Ce recueil de 40 flashcards d'excellence est conçu pour stimuler l'apprentissage actif. Chaque encadré pose une question essentielle de révision écrite ou orale sur le Chapitre 1 du livre officiel de programmation de la 4ème année secondaire en Sciences de l'Informatique (Bac Tunisie), suivie de sa réponse conforme aux exigences du Baccalauréat.

Question 1

Quelle est la différence fondamentale en informatique entre une structure de type tableau et une structure de type enregistrement ?

Réponse : Un tableau est une structure de données homogène (tous ses éléments doivent obligatoirement posséder le même type). Un enregistrement est une structure de données hétérogène (il rassemble sous un identificateur unique plusieurs variables appelées "champs" qui peuvent être de types totalement différents).

Question 2

Comment accède-t-on de manière ciblée à l'un des champs constitutifs d'une variable déclarée sous un type enregistrement ?

Réponse : On accède à un champ d'un enregistrement en utilisant la **notation pointée** : le nom de la variable d'enregistrement, suivi d'un point, puis le nom du champ cible (Exemple : élève.nom).

Question 3

Qu'est-ce qu'une déclaration d'enregistrement imbriqué et comment s'effectue l'accès à un sous-champ de cette structure ?

Réponse : C'est un enregistrement dont l'un des champs (au moins) est lui-même de type enregistrement. Pour y accéder, on enchaîne la notation pointée (Exemple : élève.date_naissance.année).

Question 4

Quel est l'apport fonctionnel et syntaxique de l'instruction globale d'ouverture "Avec .. Faire" ("With .. Do" en Pascal) ?

Réponse : L'instruction **Avec .. Faire (With .. Do** en Pascal) permet d'éviter la répétition fastidieuse du nom de la variable d'enregistrement lors de l'écriture d'instructions consécutives d'affectation, de lecture ou d'écriture en accédant directement aux champs par leurs noms.

Question 5

À quelle condition est-il formellement autorisé d'effectuer une affectation globale directe entre deux variables enregistrements ?

Réponse : L'affectation directe de l'ensemble d'une variable enregistrement dans une autre (Exemple : $A \leftarrow B$) est autorisée si et seulement si les deux variables possèdent exactement le même type d'enregistrement déclaré sous un même identificateur.

Question 6

Est-il possible de déclarer un fichier informatique dont les composants individuels sont des enregistrements personnalisés ?

Réponse : Oui, un fichier peut être déclaré comme ayant pour composants un type enregistrement personnalisé (Exemple : Fichier de Fiche). C'est le modèle type pour concevoir une base de données locale permanente et persistante.

Question 7

Quelle est la différence fondamentale entre la mémoire vive (RAM) d'un ordinateur et sa mémoire secondaire (disque) ?

Réponse : La mémoire vive (RAM) est volatile (elle s'efface dès l'arrêt de l'ordinateur ou la coupure de courant) et sa capacité globale est physiquement limitée par rapport à celle de la mémoire secondaire (disque dur, clé USB) qui, elle, est persistante.

Question 8

Pourquoi la taille physique d'un fichier sur disque est-elle qualifiée de dynamique par opposition à celle d'un tableau ?

Réponse : La taille d'un tableau est déclarée de manière statique au début du programme et ne peut être modifiée en cours d'exécution. La taille d'un fichier est dynamique et s'accroît automatiquement à chaque fois qu'on y enregistre de nouvelles données logiques.

Question 9

Quelles sont les quatre étapes logiques indispensables et obligatoires pour manipuler un fichier du début à la fin ?

Réponse : Les quatre étapes indispensables sont : 1. **L'Association** (Assign) ; 2. **L'Ouverture** (Reset ou Rewrite) ; 3. **Le Traitement** (Read ou Write) ; 4. **La Fermeture** (Close).

Question 10

Quel est l'effet technique précis de l'appel à la procédure d'association logique et physique "Associer" ("Assign" en Pascal) ?

Réponse : La procédure d'association (**Associer / Assign**) lie une variable de fichier logique (déclarée dans le code du programme) au fichier physique correspondant stocké avec son chemin complet sur un support physique externe de stockage.

Question 11

Quelle est la distinction technique majeure entre l'ouverture d'un fichier via "Reset" et son ouverture via "Rewrite" ?

Réponse : **Reset** ouvre un fichier existant en lecture/écriture sans effacer son contenu et place le pointeur au début (index 0). **Rewrite** crée un fichier à neuf : s'il n'existait pas, il est créé ; s'il existait déjà, l'intégralité de son ancien contenu est définitivement écrasée.

Question 12

Pourquoi l'absence de fermeture finale d'un fichier via la commande "Close" pose-t-elle un risque grave pour la persistance des données ?

Réponse : La fermeture d'un fichier via **Close** garantit que toutes les données temporairement accumulées dans les zones de tampons de la RAM soient physiquement et définitivement écrites sur le disque dur, et libère les descripteurs de fichiers système associés.

Question 13

Comment est structurée l'indexation de position de tous les composants logiques d'un fichier binaire d'accès direct ?

Réponse : Dans un fichier à accès direct en informatique (comme en Pascal), l'indexation de tous les blocs physiques de composants commence obligatoirement à l'adresse de base 0 et se termine à l'indice de fin égal à `FileSize(F) - 1`.

Question 14

Quel comportement l'environnement système adopte-t-il par défaut si une anomalie d'E/S survient sans programmation défensive ?

Réponse : Le programme s'interrompt brutalement sur une erreur système de type « Erreur d'Entrée/Sortie (I/O Error) », à moins que les directives de compilation de programmation défensive n'aient désactivé le contrôle automatique des erreurs d'E/S.

Question 15

Comment fonctionnent les directives de compilation locales d'E/S "\$I-" et "\$I+" sous l'environnement de développement Pascal ?

Réponse : La directive `{SI-}` désactive la gestion par défaut des erreurs fatales d'Entrée/Sortie par l'environnement d'exécution, permettant de gérer les anomalies manuellement dans le code. `{SI+}` réactive le comportement standard d'interruption.

Question 16

Quelle fonction d'évaluation système permet d'inspecter le succès ou l'échec de la dernière E/S effectuée sous l'effet de "\$I-" ?

Réponse : C'est la variable système globale **IOResult**. Elle retourne un entier égal à **0** si la dernière instruction d'E/S s'est déroulée avec succès, et un nombre d'erreur non nul (par exemple 2 si le fichier est introuvable) en cas d'anomalie.

Question 17

À quel moment précis de la lecture séquentielle d'un fichier la fonction booléenne système "Eof(F)" retourne-t-elle la valeur VRAI ?

Réponse : La fonction de fin de fichier **Eof(F)** (End of File) est une fonction booléenne qui retourne automatiquement la valeur Vrai lorsque le pointeur physique du fichier a franchi le dernier composant valide, et Faux dans le cas contraire.

Question 18

Quelle est la procédure méthodique universelle pour effectuer l'ajout propre d'un élément à la fin d'un fichier binaire structuré ?

Réponse : On déplace le pointeur de fichier à l'adresse virtuelle finale du fichier via l'instruction **Seek(F, FileSize(F))**. Tout ce qui sera écrit immédiatement après cette instruction sera enregistré à la suite des données existantes.

Question 19

Est-il possible d'exploiter de manière directe la procédure d'adressage dynamique "Seek" sur un fichier déclaré comme "Text" ?

Réponse : Non. La commande **Seek** (qui permet le déplacement direct du pointeur par index) ne s'applique que sur les fichiers à accès direct (fichiers de données typés). Elle est interdite sur un fichier texte qui est purement séquentiel.

Question 20

Comment se définit exactement un fichier de type "Text" (fichier texte séquentiel) en termes de structure de caractères ?

Réponse : Un fichier texte est un fichier composé uniquement d'une suite de caractères séquentiels encodés en texte brut (ASCII/ANSI), structuré en lignes de longueurs diverses terminées par les caractères de saut de ligne (CR-LF).

Question 21

Quelle est la syntaxe formelle de déclaration en Pascal d'une variable logique désignée comme fichier de type texte pur ?

Réponse : On la déclare dans la partie variable du programme principal en lui affectant l'identificateur de type système prédéfini **Text** (Exemple : `VAR fichier_txt : Text;`).

Question 22

Quel comportement et quelle valeur retourne la fonction d'évaluation booléenne spécifique "EoLn" sur un fichier texte ?

Réponse : La fonction **EoLn(Ft)** (End of Line) est une fonction booléenne spécifique aux fichiers textes qui retourne Vrai si le pointeur de fichier est parvenu à la marque de fin de ligne courante (caractères CR-LF), et Faux sinon.

Question 23

Quelle instruction spécifique remplace le **ReWrite** pour ouvrir un fichier texte existant en préservant le texte tout en pointant à la fin ?

Réponse : C'est la procédure d'ouverture d'extension de fichier **Append(Ft)**. Elle ouvre le fichier de texte existant en écriture, et positionne automatiquement le pointeur à la fin pour permettre la saisie de nouvelles lignes de caractères.

Question 24

Quelles différences conceptuelles fondamentales distinguent les opérations d'E/S "Read" et "ReadLn" sur un fichier texte ?

Réponse : **Read** lit les caractères ou variables demandés et maintient le pointeur de lecture sur le caractère suivant dans la ligne en cours. **ReadLn** lit la totalité de la ligne jusqu'à la marque de fin de ligne et déplace le pointeur au début de la ligne suivante.

Question 25

Comment l'utilisation de l'opération "Write" se distingue-t-elle de "WriteLn" lors de l'enregistrement de texte sur disque ?

Réponse : **Write** écrit le contenu spécifié dans le fichier sans ajouter de caractère invisible à la suite. **WriteLn** écrit le contenu et ajoute immédiatement la marque de saut de ligne de fin de ligne (CR-LF), créant un retour à la ligne automatique.

Question 26

Quelle procédure permet la suppression définitive d'un fichier physique et quelle est son exigence préalable absolue ?

Réponse : On utilise la procédure d'administration système **Erase(F)**. Le fichier associé doit être préalablement fermé sur le système avant de pouvoir exécuter la procédure d'effacement physique sur le disque.

Question 27

Comment procède-t-on pour modifier le nom physique d'un fichier enregistré et quelle en est la condition système ?

Réponse : On utilise la procédure **Rename(F, 'nouveau_nom.dat')**. Tout comme pour **Erase**, le fichier associé doit être fermé lors de l'application de la commande de changement de nom physique.

Question 28

Quelle est la différence fondamentale entre l'organisation conceptuelle d'un fichier et son mode d'accès technique ?

Réponse : L'organisation qualifie l'agencement physique et la disposition réelle des blocs d'informations sur le support matériel lors de la création. Le mode d'accès qualifie la méthode (séquentielle ou directe) d'accès du pointeur aux blocs d'informations.

Question 29

Quelle condition mathématique stricte le pointeur doit-il respecter avant toute exécution de l'instruction "Seek" ?

Réponse : Il faut vérifier rigoureusement que la position cible N saisie respecte l'inéquation : $0 \leq N < \text{FileSize}(F)$. Si $N \geq \text{FileSize}(F)$, la commande Seek tenterait de pointer en dehors des limites physiques réelles du fichier.

Question 30

Qu'est-ce que représente la notion absolue de "pointeur de fichier" du point de vue logique de la machine ?

Réponse : C'est un curseur système virtuel maintenu par le système d'exploitation, qui contient l'adresse (index) du prochain enregistrement logique à écrire ou à lire au cours d'une opération d'Entrée/Sortie.

Question 31

Est-ce que l'exécution consécutive d'une lecture via "Read" modifie d'elle-même la position de repérage du pointeur ?

Réponse : Oui. L'exécution de l'opération `Read(F, variable)` lit la donnée à la position courante, puis déplace automatiquement le pointeur de fichier sur le bloc immédiatement suivant ($\text{index} + 1$).

Question 32

Pourquoi la structure de fichier binaire d'accès direct s'avère-t-elle particulièrement efficace pour les opérations de mise à jour ?

Réponse : Parce que les fichiers binaires typés sont à accès direct : ils permettent un pointage arbitraire instantané. Pour faire un ajout, il suffit de combiner `Seek(F, FileSize(F))` suivi d'une commande classique de type `Write(F, valeur)`.

Question 33

Quel effet l'imbrication redondante de structures "Avec .. Faire" multiples et relatives à une même variable a-t-elle sur le compilateur ?

Réponse : C'est techniquement inutile et redondant, mais cela n'entraîne pas d'erreur de compilation. Le compilateur privilégiera directement la portée locale associée à la variable d'enregistrement ouverte.

Question 34

Est-il permis d'inclure un composant de type fichier logique comme champ au sein d'une structure d'enregistrement ?

Réponse : Non. Les fichiers sont des structures gérées par le système d'exploitation en mémoire secondaire et ne peuvent être inclus comme champs d'un enregistrement en mémoire vive.

Question 35

Comment doit s'organiser méthodiquement le tri des enregistrements d'un fichier informatique pour être réalisable ?

Réponse : Non, pour trier des données logiques de manière efficace, on doit d'abord lire séquentiellement les données du fichier pour remplir un tableau en mémoire vive, trier ce tableau, puis réécrire le tableau trié dans le fichier d'origine.

Question 36

L'appel d'ouverture physique "Reset" positionne-t-il directement le statut système d'un fichier à Eof = VRAI ?

Réponse : Non, elle retourne la valeur Faux. Après un Reset, le pointeur est toujours positionné au tout début (index 0). Elle ne retournera Vrai qu'après la lecture consécutive du cinquième et dernier élément stocké.

Question 37

Quel est l'effet technique précis de l'appel à la procédure de troncature sélective d'un fichier "Truncate" ?

Réponse : La commande **Truncate(F)** supprime instantanément tous les composants du fichier situés à partir de la position courante du pointeur, réduisant ainsi la taille finale physique du fichier.

Question 38

Quelle fonction système standard, extérieure aux fichiers, permet la création dynamique de dossiers sur le disque ?

Réponse : Bien que ne faisant pas partie du chapitre direct, cela s'effectue grâce à la commande système d'administration de fichier standard appelée **MkDir('C: chemin')**.

Question 39

Quel est le type de données numérique de retour strict partagé par les fonctions "FileSize", "FilePos" et "IOResult" ?

Réponse : Ces trois fonctions standard d'évaluation système retournent obligatoirement des types de données numériques entiers (**Longint** ou **Integer** en Pascal).

Question 40

Quelle raison justifie le fait que l'écriture au sein d'un fichier texte soit quantifiée octet par octet par le système ?

Réponse : Parce que chaque caractère composant le fichier texte est converti et enregistré sous la forme d'un code numérique de 1 octet (valeurs de 0 à 255) représentant des symboles de police de caractères lisibles.

Chapter 3

Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo

3.1 Le Quiz d'Évaluation (10 Questions Types Bac)

Ce quiz est conçu pour évaluer de manière rigoureuse vos compétences théoriques et pratiques sur le Chapitre 1 du programme tunisien.

Question 1

Soit la variable d'enregistrement déclaré sous le type Fiche. On souhaite copier l'intégralité des champs de l'étudiant A vers l'étudiant B. Quelle instruction est correcte ?

- A) $B.champs \leftarrow A.champs$
- B) $B \leftarrow A$ (si de même type déclaré)
- C) $B.nom \leftarrow A.nom$ et copier champ par champ uniquement.

Correction détaillée : La bonne réponse est la **B**. En algorithmique comme en Pascal, si deux variables enregistrement possèdent exactement le même type déclaré, on peut copier globalement la totalité de la structure en une seule opération d'affectation globale directe sans devoir copier manuellement chaque champ individuellement.

Question 2

VRAI ou FAUX : Lors de la lecture d'un fichier binaire d'accès direct, l'exécution consécutive de l'opération de lecture $Read(F, V)$ maintient le pointeur de fichier sur le même index.

Correction détaillée : **FAUX**. En informatique, toute lecture ou écriture valide via les procédures $Read$ ou $Write$ déplace automatiquement le curseur du pointeur physique de l'index courant i vers l'index suivant $i + 1$.

Question 3

Quelle valeur retourne la fonction booléenne $Eof(F)$ lorsque le pointeur est positionné au tout début d'un fichier vide qui vient d'être initialisé avec $Rewrite$?

- A) Faux
- B) Vrai
- C) Erreur système

Correction détaillée : La bonne réponse est la **B**. Un fichier vide nouvellement créé ne contient aucun élément. Le pointeur is donc immédiatement positionné sur la marque finale de fin de fichier, ce qui implique que la fonction Eof retourne instantanément Vrai.

Question 4

VRAI ou FAUX : L'utilisation de l'instruction Seek est autorisée sur les fichiers de type Text.

Correction détaillée : **FAUX**. Les fichiers textes sont de type séquentiel pur. Les lignes possédant des longueurs variables en termes de caractères, il est impossible pour le système de calculer une adresse directe. L'usage de la commande Seek est strictement réservé aux fichiers binaires d'accès direct.

Question 5

Quelle est la valeur de retour de la fonction système IOResult lorsque l'opération d'Entrée/Sortie précédente s'est exécutée avec un succès total ?

- A) -1
- B) 0
- C) 1

Correction détaillée : La bonne réponse est la **B**. En Pascal, le code de retour **0** signifie « Aucune erreur rencontrée ». Tout nombre entier non nul retourné par la fonction IOResult est un identificateur d'erreur système.

Question 6

VRAI ou FAUX : L'exécution de la procédure Reset vide automatiquement le contenu d'un fichier physique stocké sur le disque pour le réinitialiser.

Correction détaillée : **FAUX**. C'est la procédure Rewrite qui vide ou écrase le fichier existant. La procédure Reset ouvre le fichier existant en lecture ou en écriture tout en conservant scrupuleusement l'intégralité de son contenu.

Question 7

Quels caractères invisibles codifient la fin d'une ligne physique dans un fichier Texte ?

- A) La lettre 'Z'
- B) La marque de retour chariot et saut de ligne (CR-LF)
- C) Un espace vide

Correction détaillée : La bonne réponse est la **B**. Dans le modèle de fichier texte séquentiel, chaque fin de ligne est matérialisée par la présence du couple de caractères de contrôle ASCII invisibles Carriage Return (Retour Chariot - ASCII 13) et Line Feed (Saut de Ligne - ASCII 10).

Question 8

VRAI ou FAUX : L'oubli de la fermeture Close(F) d'un fichier ouvert en écriture dans un programme n'a aucune conséquence sur les données enregistrées.

Correction détaillée : **FAUX**. Si le fichier n'est pas correctement fermé, les dernières données accumulées temporairement dans le tampon mémoire vive (RAM) risquent d'être totalement perdues, et le descripteur système du fichier restera anormalement verrouillé.

Question 9

Soit un fichier d'accès direct de taille de 15 éléments. Quel est l'index maximal autorisé lors d'un Seek ?

- A) 15
- B) 14
- C) 16

Correction détaillée : La bonne réponse est la **B**. L'indexation commençant à l'adresse de base **0**, les index autorisés pour les éléments existants d'un fichier de 15 éléments s'étendent de **0** à **14**. Pointer à l'index 15 sert exclusivement à effectuer un ajout de composant à la fin.

Question 10

VRAI ou FAUX : Une variable de type enregistrement peut être écrite dans un fichier en une seule instruction d'écriture.

Correction détaillée : **VRAI**. On peut écrire d'un seul coup tout un enregistrement typé via `Write(F, variable_fiche)`, à condition que le fichier ait été déclaré comme un type contenant ces enregistrements (`File Of Fiche`).

3.2 Mémento de l'Excellence (Tableau Récapitulatif Universel)

Algorithme	Langage Pascal	Type Fichier	Rôle de l'Opération Système
Associer(F, chemin)	Assign(F, chemin);	Tous	Lie la variable de fichier logique à son chemin physique réel.
Recréer(F)	ReWrite(F);	Tous	Ouvre un nouveau fichier ou écrase un fichier existant pour écriture à neuf.
Ouvrir(F)	ReSet(F);	Tous	Ouvre un fichier existant en lecture/écriture avec pointeur au début.
Fermer(F)	Close(F);	Tous	Enregistre les données des tampons sur le disque et libère les descripteurs.
Fin_fichier(F)	Eof(F)	Tous	Fonction booléenne indiquant si le pointeur a dépassé le dernier élément.
Pointer(F, N)	Seek(F, N);	Binaire Direct	Déplace le pointeur sur le composant numéro N .
Taille_fichier(F)	FileSize(F)	Binaire Direct	Retourne le nombre total de composants dans le fichier.
Position_fichier(F)	FilePos(F)	Binaire Direct	Retourne l'index courant (0 à Taille-1) de la position du pointeur.
Fin_ligne(Ft)	EoLn(Ft)	Texte Séquentiel	Indique si le pointeur est sur la marque de saut de ligne (CR-LF).
Ajouter(Ft)	Append(Ft);	Texte Séquentiel	Ouvre un fichier texte existant en plaçant le pointeur à la fin pour ajout.

Table 3.1: Mémento de synthèse des instructions sur Fichiers

3.3 Les 5 Pièges Classiques du Bac à éviter absolument

Rappels et Mises en Garde pour l'Épreuve Pratique

- 1. Indexation commençant à 0** : Ne confondez jamais la taille physique du fichier (par exemple 10 éléments) et les index autorisés pour un `Seek` (de 0 à 9). L'appel de `Seek(F, FileSize(F))` est exclusivement réservé pour se positionner en vue d'un ajout à la fin du fichier.
- 2. L'oubli du Close** : Si vous oubliez de fermer un fichier avec la procédure `Close(F)` à la fin de vos traitements en écriture, vos modifications ne seront pas enregistrées physiquement sur le disque et le fichier restera verrouillé.
- 3. Seek interdit sur les fichiers Text** : L'utilisation de la procédure `Seek` sur un fichier déclaré de type `Text` est une erreur fatale et provoque l'échec de la compilation. Seuls les fichiers typés binaires autorisent l'accès direct.
- 4. L'utilisation d'Append** : N'essayez jamais d'utiliser `Append` sur un fichier binaire. Cette procédure est exclusivement réservée aux fichiers de type `Text` pour ouvrir en mode ajout. Pour un fichier binaire, vous devez faire `Reset(F)` suivi de `Seek(F, FileSize(F))`.

5. Initialisation avec ReWrite : Soyez vigilant ! L'appel à la procédure `ReWrite(F)` écrase instantanément et sans avertissement l'intégralité du contenu d'un fichier physique s'il existait déjà sur le disque dur.